

Data Structure and Algorithm, Spring 2026

Homework 2

Due: 13:00:00, Tuesday, April 7, 2026

TA E-mail: dsa_ta@csie.ntu.edu.tw

Rules and Instructions

- Any form of cheating, lying, or plagiarism will not be tolerated. Students can get zero scores and/or fail the class and/or be kicked out of school and/or receive other punishments for those kinds of misconducts.
- Discussions on course materials and homework solutions are encouraged. But you should write the final solutions alone and understand them fully. Books, notes, and Internet resources (including chatGPT) can be consulted, but not copied from.
- Since everyone needs to write the final solutions alone, there is **absolutely no need to lend your homework solutions and/or source codes to your classmates at any time**. In order to maximize the level of fairness in this class, lending and borrowing homework solutions are both regarded as dishonest behaviors and will be punished according to the honesty policy.
- We will detect and penalize plagiarism with the criteria of **line-by-line similarity** in any part of your code/writing, when compared with the code/writing from any student or any external resource. Exceptions will be given to those showing **time-stamped records** that the solution comes from a meaningful interactive discussion with human and/or AI tools, **rather than copying from** others.
- In Homework 2, the problem set contains 4 problems. The set is divided into two parts, the non-programming part (Problems 1, 2) and the programming part (Problems 3, 4).

- For your solutions in the non-programming part, unless otherwise noted, there are two common requirements:
 - Your answer should be as simple as possible. For instance, $\Theta(\frac{1}{n} + \lg(n^{541}) + 8763n^2)$ should be written as $\Theta(n^2)$.
 - Your answer should also come with a *brief* explanation. The word *brief* means “very short.” An example of a *brief* explanation is as follows.

Sample problem: Write down the time complexity for this algorithm using the Θ notation along with an expression of n .

HELLO-WORLD(n)

```
1  for  $i = 1 \dots 2n$ 
2      for  $j = 1 \dots 2n$ 
3          print("Hello world")
```

Sample solution: The iteration counts of the two loops are $2n$ and $2n$ respectively, so there are $4n^2$ iterations. Hence, the time complexity is $\Theta(n^2)$.

□

- For problems in the non-programming part, you should combine your solutions in *one* PDF file. Your file should generally be legible. Your solution must be as simple as possible. At the TAs' discretion, solutions which are too complicated can be penalized or even regarded as incorrect. If you would like to use any theorem which is not mentioned in the classes, please include its proof in your solution.
- The PDF file for the non-programming part should be submitted to Gradescope as instructed, and you should use Gradescope to tag the pages that correspond to each subproblem to facilitate the TAs' grading. **Failure to tag the correct pages of the subproblem can cost you a 10% penalty.**

- For problems in the programming part, you should write your code in C programming language, and then submit the code via the *DSA Judge+*: <https://dsa.csie.ntu.edu.tw/>. Each day, you can submit up to 10 times for each problem. The judge system will compile your code with

```
gcc [id].c -O2 -lm -std=c17
```

- For debugging in the programming part, we strongly suggest you produce your own test-cases with programs and/or use tools like `gdb` or compile with flag `-fsanitize=address` to help you solve issues like illegal memory usage. For `gdb`, you are strongly encouraged to use compile flag `-g` to preserve symbols after compiling.

Note: For students who use IDE (VScode, DevC++...) you can modify the compile command in the settings/preference section. Below are some examples:

```
- gcc [id].c -O2 -lm -std=c17 -fsanitize=address
                        # Works on Unix-like OS, but possibly not M1/M2
- gcc [id].c -O2 -lm -std=c17 -g # use it with gdb
```

- The score of the part that is submitted after the deadline will get some penalties according to the following rule:

$$Late\ Score = \max \left(\left(\frac{5 \times 86400 - Delay\ Time(sec.)}{5 \times 86400} \right) \times Original\ Score, 0 \right)$$

As mentioned at the beginning of the class, you have **eight gold medals** this semester, each of which can be used to extend the deadline of **one** programming problem by **24 hours**. You can choose how to allocate the medals around the end of the semester.

- If you have questions about this homework, please go to the Discord channel and discuss the issues with your classmates. This is *strongly preferred* because it will provide everyone with a more interactive learning experience. If you really need email clarifications, you are encouraged to include the following tag in the subject of your email:
 - Please include the following tag in the subject of your email: "[HW2.Px]", specifying the problem where you have questions. For example, "[HW2.P2] Is k in subproblem 5 an integer?". Adding the tag allows the TAs to track the status of each email and to provide faster responses to you.
 - If you want to provide your code segments to us as part of your question, please upload it to [Gist](https://gist.github.com) or similar platforms and provide the link. Please remember to protect your uploaded materials with proper permission settings. Screenshots or code segments directly included in the email are discouraged and may not be reviewed.

Problem 1 - Dendro Symmetrica Aequalis (100 pts)

1. (10 pts) Consider the following recursive construction of a family of perfect binary trees whose nodes take values in $0, 1$. Let $T = \text{Node}(v, L, R)$ denote a binary tree with root value v , left subtree L , and right subtree R , and let NIL represent the empty tree. Then define:

$$\text{mirror}(T) = \begin{cases} \text{NIL} & \text{if } T = \text{NIL} \\ \text{Node}(v, \text{mirror}(R), \text{mirror}(L)) & \text{if } T = \text{Node}(v, L, R) \end{cases}$$

$$\text{invert}(T) = \begin{cases} \text{NIL} & \text{if } T = \text{NIL} \\ \text{Node}(1-v, \text{invert}(L), \text{invert}(R)) & \text{if } T = \text{Node}(v, L, R) \end{cases}$$

Define the family T_n recursively.

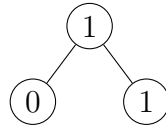
- $T_1 = \text{Node}(0, \text{NIL}, \text{NIL})$,
- $T_n = \text{Node}(1, T_{n-1}, \text{mirror}(\text{invert}(T_{n-1})))$ for $n \geq 2$.

Example — recursive construction:

- T_1 :



- $T_2 = \text{Node}(1, T_1, \text{mirror}(\text{invert}(T_1)))$:



Define S_n as the binary string representing the sequence of node values obtained from the in-order traversal of T_n . Construct and draw T_4 . Then, write down S_4 . You do not need to provide the construction steps.

2. (15 pts) Prove that for every $n \geq 2$ and every position i with $1 \leq i \leq 2^n - 1$ and $i \neq 2^{n-1}$,

$$S_n[i] + S_n[2^n - i] = 1.$$

In other words, for $n \geq 2$, the sequence S_n is **symmetric with negated bits** around its middle bit, which corresponds to the root of T_n and always has value 1. Note that we seek a formal mathematical proof, preferably one that exploits the recursive structure of T_n , rather than a heuristic or hand-waving argument.

3. (20 pts) Index the nodes of T_n according to their positions in the in-order traversal, labeling the first visited node as node 1. Then, the value of node i in T_n corresponds to the i -th bit of S_n .

For small n , one can construct T_n (or S_n) explicitly and directly query the value of any node. However, for large n (e.g., $n = 60$, which yields over 10^{18} nodes), explicit construction is computationally infeasible. Therefore, we ask you to design an efficient algorithm to determine the value of any node i with $1 \leq i \leq 2^n - 1$ **without constructing T_n or S_n** .

In particular, your algorithm should have a worst-case time complexity of $O(n)$. Express your algorithm with pseudocode with ≤ 30 lines, then briefly explain why your algorithm is correct and why it meets the required time complexity with another ≤ 30 lines.

4. (10 pts) With the definitions above, let $f(n, k)$ denote the value (either 0 or 1) of the k -th node in T_n under in-order traversal. Define

$$P(n, k) = \sum_{i=1}^k f(n, i),$$

which counts the number of nodes with value 1 among the first k nodes in the in-order traversal of T_n , for $1 \leq k \leq 2^n - 1$. Derive a recursive formula for $P(n, k)$ in terms of $P(n - 1, \bullet)$. The derivation is expected to be mathematically rigorous.

Hint: You may use the following property without proof. Let $W(n) = P(n, 2^n - 1)$, the total number of 1-valued nodes in T_n . Then, $W(1) = 0$ and $W(n) = 2^{n-1}$ for $n \geq 2$.

5. (20 pts) Use your result in the previous sub-problem to design an algorithm that computes any $P(n, k)$ within $O(n)$ time. Express your algorithm with pseudocode with ≤ 30 lines, then briefly explain why your algorithm is correct and why it meets the required time complexity with another ≤ 30 lines.

6. (25 pts) Given the k_1 -th node and the k_2 -th node in T_n , compute the Path-XOR of them, as defined below.

In particular, in a tree, there is a unique path between any two nodes.

Definition 1 (Path). For two nodes k_1 and k_2 in T_n , the path between them is the unique sequence of nodes

$$k_1 = x_0, x_1, \dots, x_m = k_2$$

such that each consecutive pair x_j, x_{j+1} is connected by an edge.

Definition 2 (Path-XOR). For two nodes k_1 and k_2 in T_n that is connected by a unique path

$$k_1 = x_0, x_1, \dots, x_m = k_2,$$

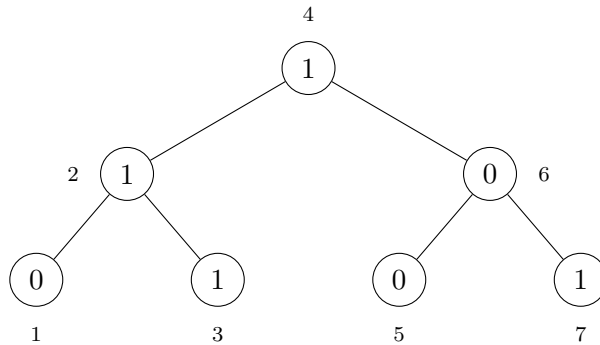
their Path-XOR is

$$\text{PathXOR}(n, k_1, k_2) = \bigoplus_{j=0}^m f(n, x_j).$$

Given (n, k_1, k_2) with $n \geq 2$ and $1 \leq k_1 < k_2 \leq 2^n - 1$, design an algorithm that computes $\text{PathXOR}(n, k_1, k_2)$ in T_n in $O(n)$ time. Given the time constraint, you likely cannot construct the tree explicitly.

Express your algorithm with pseudocode with ≤ 40 lines, then briefly explain why your algorithm is correct and why it meets the required time complexity with another ≤ 40 lines.

Example: Consider the following T_3 , where each node is indexed by its position in the in-order traversal.



- $\text{PathXOR}(3, 1, 6)$: path = $1 \rightarrow 2 \rightarrow 4 \rightarrow 6$, values = $0, 1, 1, 0$, XOR = 0 .
- $\text{PathXOR}(3, 4, 5)$: path = $4 \rightarrow 6 \rightarrow 5$, values = $1, 0, 0$, XOR = 1 .

Hint: For two nodes k_1 and k_2 in T_n , define their lowest common ancestor (LCA) u as the unique deepest node on their path that is an ancestor of both, where a node is considered an ancestor of itself. It may be worth considering the relationship between $\text{PathXOR}(n, k_1, k_2)$, $\text{PathXOR}(n, k_1, u)$, and $\text{PathXOR}(n, u, k_2)$.

Problem 2 - Das Studenten-Abenteuer (100 pts)

1. (20 pts) A station master operates a T-shaped railway consisting of three dead-end tracks: S_1 , S_2 , and S_3 . Since each track is a dead end, train cars can only be moved in a Last-In-First-Out manner; thus, each track functions as a stack.

Initially, n train cars, labeled 1 through n in arbitrary order, are placed on track S_1 , while tracks S_2 and S_3 are empty.

The station master can carefully route the train cars using the central junction. However, due to the track design, only the following four operations are allowed:

- (a) Move the front car from S_1 to S_2 (`S1.pop()` -> `S2.push()`)
- (b) Move the front car from S_1 to S_3 (`S1.pop()` -> `S3.push()`)
- (c) Move the front car from S_2 to S_1 (`S2.pop()` -> `S1.push()`)
- (d) Move the front car from S_2 to S_3 (`S2.pop()` -> `S3.push()`)

No other operations can be performed. Design an algorithm that sorts the n cars into the stack (track) S_3 in $O(n^2)$ time, where car 1 is at the top of the stack and car n is at the bottom of the stack. Express your algorithm with pseudocode with ≤ 20 lines, and then briefly explain why it meets the required time complexity with another ≤ 30 lines. You do not need to explain why your algorithm is correct.

2. (30 pts) Consider a computer system equipped with two independent processors, labeled 0 and 1. Each processor maintains its own ready queue, where processes wait to be executed by that specific CPU.

You are given a sequence of n processes in strictly increasing order of arrival time. The i -th process arrives at time t_i and requires c_i units of uninterrupted CPU execution time.

The system follows the scheduling rules and assumptions below:

- At each arrival time t_i , the operating system examines the current state of both ready queues and assigns the new process to the queue with fewer pending processes (measured by the number of waiting processes).
- If the two queues contain the same number of waiting processes, the process is assigned to the ready queue of Processor 0.
- If a process completes exactly at time t_i , it is removed from its queue before the new process is assigned.

- There is no overhead for context switching or for dispatching a process to an idle processor.

Design an algorithm that computes the exact completion time of each of the n processes. Your algorithm must be completed with $O(n)$ time and $O(n)$ extra-space complexity. Express your algorithm with pseudocode with ≤ 40 lines, and then briefly explain why your algorithm is correct and why it meets the required time and extra-space complexity with another ≤ 40 lines.

3. (25 pts) Next, we will provide you with a story version, and an equivalent mathematical version. Please feel free to decide what version(s) you want to read to solve the subproblem.

Story version: After the intense DSA class, the freshmen are eager to unwind. A group of first-year students heads to the famous Gongguan Night Market, walking sequentially from stall 1 to stall n . As they sample the food, they assign a deliciousness score $A[i]$ to the i -th stall on the spot.

However, they cannot remember every stall they visit. A stall remains in memory only if it leaves a **deep impression**. The rule governing such impressions is simple: an older (more distant) stall is remembered only if its deliciousness score is strictly higher than those of all newer stalls.

Whenever the student visits a new stall at position i , they compare it with the stalls currently in memory. If any previously remembered stall has a score less than or equal to $A[i]$, it is immediately forgotten. This is because the newer stall is at least as tasty and thus completely overshadows the older one.

To formalize this process, let S be a dynamic sequence representing the student's memory, storing the indices of stalls that remain remembered. As the student visits each stall i (for $i = 1, 2, \dots, n$), the sequence S is updated according to the **deep impression** rule.

Mathematical version: Given an array A of n integers, we iterate through the array from index $i = 1$ to n . At each step i , we dynamically maintain a sequence S of indices.

At any current step i , the sequence S contains a specific subset of indices from $\{1, 2, \dots, i\}$. An index j (where $j \leq i$) is kept in S if and only if its value $A[j]$ is strictly greater than all subsequent elements up to index i . Mathematically, an index $j \in S$ at step i if and only if:

$$A[j] > A[k] \quad \text{for all } k \text{ such that } j < k \leq i.$$

Let $|S| = \ell$. Design an algorithm to maintain an array s such that $s[x] = S_x$ for $1 \leq x \leq \ell$ at any step i from 1 to n . Your algorithm must take a total of $O(n)$ time as you scan the array A . Formally, denote $T(i)$ as the time spent in step i , the algorithm must satisfy $\sum_{i=1}^n T(i) = O(n)$. Express your algorithm with pseudocode with ≤ 15 lines, and then briefly explain why your algorithm is correct and why it meets the required time complexity with another ≤ 30 lines.

4. (25 pts) Similar to the previous subproblem, we will provide you with a story version, and an equivalent mathematical version. Please feel free to decide what version(s) you want to read to solve the subproblem.

Story version: As the group of students stroll through the night market, curious passersby occasionally stop to ask for recommendations. Notably, each question concerns a range that ends exactly at the student's *current* location.

At the precise moment the student arrives at stall i and updates their memory sequence S , the k -th passerby approaches and asks: “If I only want to visit stalls from L_k up to your current stall R_k , what is the maximum deliciousness score I can obtain?”

Because these questions arise sequentially as the student moves from stall 1 to n , the q queries are naturally ordered by non-decreasing right endpoints R_k .

From the previous subproblem, when $i = R_k$, you have access to the current memory sequence S . Given a query for the maximum over the interval $[L_k, R_k]$, how can you leverage the indices stored in S to efficiently determine the answer?

Mathematical version: Suppose we are given q queries $(L_1, R_1), (L_2, R_2), \dots, (L_q, R_q)$ such that $R_1 \leq R_2 \leq \dots \leq R_q$. Compute the maximum within the subarray $A[L_k \dots R_k]$ for each query.

Design an algorithm for answering all q queries together efficiently. You are required to implement your solution using the array s defined in the previous problem. The complete algorithm, including both maintaining the array s and processing all q queries, must run in $O(n + q \log n)$ time and use $O(n)$ additional space. Express your algorithm with pseudocode with ≤ 40 lines, and then briefly explain why your algorithm is correct and why it meets the required time and extra-space complexity with another ≤ 60 lines. You are strongly encouraged to include any properties that you observe in $(s[1], \dots, s[\ell])$ and $(A[s[1]], \dots, A[s[\ell]])$ in your explanation.

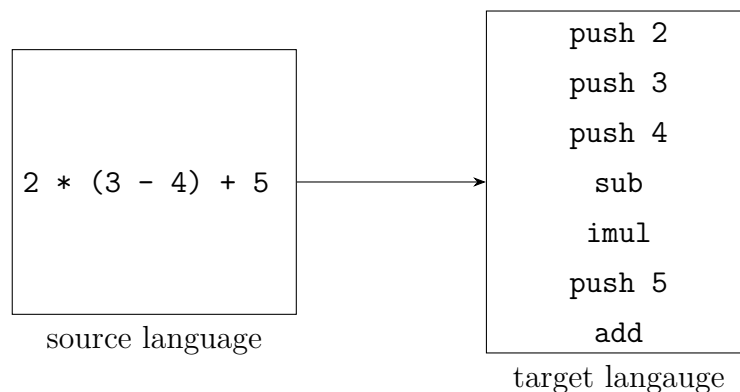
P.S. A Note on Algorithmic Problem Solving: In general, queries in algorithmic problems are *not* presented in a conveniently sorted order. A powerful and widely used technique is to read all queries in advance and sort them manually by their right endpoints before processing. This approach, known as **offline processing**, enables you to apply the same sweeping algorithm you have just developed to solve what might initially appear to be a much more challenging problem.

Furthermore, this particular Range Maximum Query problem can also be handled in an online manner using advanced data structures that you will learn as you progress in your algorithmic journey.

Problem 3 - Display Stack Arithmetic (100 pts)

Problem Description

In this homework, you will design an expression evaluator that converts infix expressions into a sequence of assembly-like instructions. This task mainly involves the Infix-to-Postfix algorithm introduced in the class.



There are two types of expressions in the input.

1. Basic expression **expr**: Contains integers, arithmetic operators, and parentheses, as shown in Table 2.1.
2. Conditional Expression: The format (**if** **<expr1>** **then** **<expr2>** **else** **<expr3>**). Note that the conditional expression is guaranteed to be enclosed in parentheses, and all **<expr1>**, **<expr2>**, **<expr3>** are guaranteed to be basic expressions. That is, there will not be nested **if** (yeah!!).

To achieve the main goal of converting an expression to an assembly-like language, your program should translate the input expression into postfix form based on Table 2.2.

When a conditional expression block (**if** **<expr1>** **then** **<expr2>** **else** **<expr3>**) is encountered, the instructions must follow this specific order.

1. Instructions for **<expr1>**.
2. **jz m k**. Here k is the total number of instructions inside both the **then** and **else** blocks combined, including their respective **ret** instructions. And m is the number of instructions to skip if the condition is not met, i.e., the length of the **then** block, including its **ret**.

Priority	Operator	Description	Associativity
1	()	Parentheses	N/A
2	if-then-else	Conditional Expression	N/A
3	^	Exponential	Right
4	*, /	Multiplication, Integer Division	Left
5	+, -	Addition, Subtraction	Left
6	<=	Comparison	Left

Table 2.1: Precedence and Associativity Table (from highest to lowest precedence)

Input Token	Output Instruction
number	push number
^	pow
*	imul
/	idiv
+	add
-	sub
<=	setle

Table 2.2: Token to Instruction Mapping

3. Instructions for <expr2>, followed by a **ret** instruction.
4. Instructions for <expr3>, followed by a **ret** instruction.

Input

The input consists of a single line of length ℓ , which may contain a combination of basic and conditional expressions. All tokens (integers, operators, **if**, **then**, and **else**) are guaranteed to be separated by a single space.

Output

Output the sequence of instructions in postfix form.

Constraints

- $1 \leq \ell \leq 10^6$
- The input expression is guaranteed to be a valid infix expression defined above.
- All numbers n in the input expression satisfy $1 \leq n \leq 10^9$.
- The necessary number of expressions for the **if-then-else** expressions satisfy $1 \leq m, k \leq 1000$.

Subtasks

Subtask 1 (30 pts)

Input does not contain **if**, **<=**, or **^**.

Subtask 2 (30 pts)

Input does not contain **if**-statement.

Subtask 3 (40 pts)

No other constraints.

Sample Testcases

Sample Input 1

1 + 5 * 8 / 7 - 8

Sample Output 1

```
push 1
push 5
push 8
imul
push 7
idiv
add
push 8
sub
```

Sample Input 2

$3 \wedge 4 \wedge 5$

Sample Output 2

push 3
push 4
push 5
pow
pow

Sample Input 3

$3 + ((\text{if } 5 \leq 10 \text{ then } 5 + 5 \text{ else } 3 \wedge 2 \wedge 4) - 2)$

Sample Output 3

push 3
push 5
push 10
setle
jz 4 10
push 5
push 5
add
ret
push 3
push 2
push 4
pow
pow
ret
push 2
sub
add

Problem 4 - Devine Sweetmelon Agency (100 pts)

Problem Description

We will provide a story version followed by an equivalent mathematical version. Please feel free to decide what version(s) you want to read to solve this problem.

Story version

Devine Sweetmelon Agency (a.k.a. DSA) is an internationally renowned conglomerate specializing in the sale of sweet melons. Its corporate structure comprises N companies, labeled 1 through N . Each company i has exactly one parent company x_i , except for the topmost headquarter (company 1), which has no parent. Such a tree structure rooted at company 1 guarantees that the ownership structure is acyclic, ensuring that every company can trace its lineage up to company 1. Each company i sells its premium melons at a base price of P_i .

FRUITY has a strong fondness for sweet melons and aims to consume as many as possible each day over the next M days. However, FRUITY has a daily budget of C dollars, which does not carry over to subsequent days. To maximize the number of melons purchased, FRUITY closely monitors the market. Each day, several companies may announce new discount events. Under DSA's standard policy, once a discount event becomes effective, it remains valid for exactly K days.

A discount event announced on day j for company b is characterized by a delay of f days and a discount value of g . The discount becomes active on day $j + f$ and remains valid through day $j + f + K - 1$. If multiple discount events for the same company are active on a given day, FRUITY always applies the one with the largest discount value g . Accordingly, the final price of a melon from company b is given by $P_b - g_{\max}$, where $g_{\max}$ denotes the maximum active discount for that company on that day.

To simplify the purchasing process, FRUITY signs a contract with **exactly one company** in DSA each day. Under this contract, FRUITY must purchase **exactly one sweet melon** from the chosen company and from each of its descendant companies on that day. Given the daily budget constraint and the evolving discounts in the market, determine the maximum number of sweet melons that FRUITY can purchase on each day.

The story version is equivalent to the following one for those who do not like fruits. Consider a hierarchical structure consisting of N nodes labeled 1 through N , where each node i except

for $i = 1$ has exactly one designated parent node x_i , ensuring the structure to be a tree rooted at $i = 1$. Each node i has an initial base weight P_i .

Over a sequence of M consecutive days, a series of modification events occurs. Each event, announced on day j for a target node b , is specified by a delay f and a discount value g . The event becomes active starting on day $j + f$ and remains valid until the end of day $j + f + K - 1$.

On any given day d , the effective weight of a node i is computed as

$$W_i(d) = P_i - g_{\max}(i),$$

where $g_{\max}(i)$ is the maximum discount among all active events for node i on day d , and defaults to 0 if no events are active.

A *valid hierarchical group* is defined as a node along with all its direct and indirect descendants. Your task is to process the overlapping interval events and, for each day d ($1 \leq d \leq M$) and for a given constant C , determine the maximum number of nodes in a valid hierarchical group such that the sum of their effective weights does not exceed C .

Input

The inputs are described with the story version, while their notations are exactly the same as the story-less version.

- The first line contains four space-separated integers N , M , K , and C , representing the number of companies, the number of days in the simulation, the fixed duration of each discount event, and FRUITY's daily budget, respectively.
- The second line contains $N - 1$ space-separated integers $x_2, x_3, \dots, x_N$, where x_i represents the parent company of company i .
- The third line contains N space-separated integers $P_1, P_2, \dots, P_N$, where P_i represents the base prices of melons of company i .

The remaining input is divided into M sections, corresponding to the market announcements for days 1 through M . Each section begins with a line containing a single integer A_j , denoting the number of new discount events announced on day j . Each of the next A_j lines contains three integers b , f , and g , representing the company ID, the delay before the discount becomes active, and the discount value, respectively.

Output

The output should contain M lines, where the j -th line indicates the maximum number of sweet melons FRUITY can purchase on day j , or equivalently, the maximum number of nodes in a

valid hierarchical group such that the sum of their effective weights does not exceed C . If no hierarchical group's total effective weight fits within C (i.e. FRUITY cannot buy any melons), output 0 for that day.

Constraints

- $2 \leq N \leq 10^6$
- $1 \leq M \leq 10^6$
- $2 \leq N \times M \leq 10^6$
- $1 \leq K \leq M$
- $1 \leq C \leq 10^9$
- $1 \leq x_i \leq N$ and $x_i \neq i$
- $1 \leq P_i \leq 10^9$
- $1 \leq \sum A_j \leq 10^6$
- $1 \leq b \leq N$
- $0 \leq f \leq M - j$ (Every discount event will start within the M simulated days)
- $1 \leq g \leq 10^9$
- For any company i and any of its valid discounts g , $P_i - g \geq 1$.

Subtasks

Subtask 1 (15 pts)

- $N \times M \leq 10^3$
- $\sum A_j \leq 10^3$

Subtask 2 (20 pts)

$x_i = i - 1$. That is, the structure is special.

Subtask 3 (25 pts)

$K = M$.

Subtask 3 (25 pts)

No additional constraints.

Sample Testcases

Sample Input 1

5 3 2 100
3 1 3 4
66 24 22 69 56
10
1 2 15
3 2 8
2 2 22
3 1 2
2 1 20
4 1 30
4 2 5
1 2 6
1 0 35
5 1 21
1
1 1 33
4
4 0 24
1 0 17
2 0 5
5 0 42

Sample Output 1

1
4
4

Sample Input 2

3 5 3 10
1 1
10 10 10
9
1 0 4
2 4 6
3 0 4
2 3 6
1 0 8
2 4 7
1 1 5
1 4 4
2 1 7
1
3 2 8
2
3 1 7
2 1 3
7
1 1 4
1 1 1
1 0 2
2 0 3
2 1 2
2 0 5
1 1 1
1
1 0 3

Sample Output 2

1
1
1
3
1

Hint

Note that if the data for valid discount/modification events is maintained properly, their expiration times inherently follow a First-In-First-Out (FIFO) order. Leveraging this property can potentially help in designing a more efficient algorithm.